Conversão por Varredura de Primitivas Básicas

Computação Gráfica

Prof. Robson Pequeno de Sousa

Matemática Básica

Equação da Reta

Dados dois pontos $P_1(X_1,Y_1)$, $P_2(X_2, Y_2)$.

Considere um terceiro ponto sob a reta:

$P = (X,Y)$

Declive $= (Y_2 - Y_1)/(X_2 - X_1)$

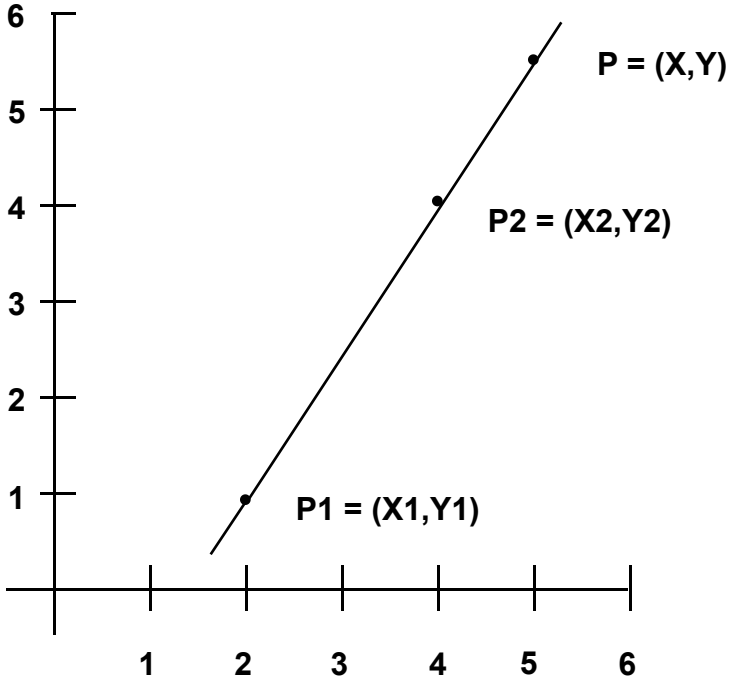
$= (Y - Y_1)/(X - X_1)$

Resolvendo para Y

$Y = [(Y_2-Y_1)/(X_2-X_1)]*(X-X_1)+ Y_1$

$Y = mx + b$

Sistema de coordenadas cartesianas



$$m = \frac{Y2-Y1}{X2-X1}$$

Fórmulas Úteis

- **O comprimento do segmento de reta entre P_1 e P_2**

  $$L = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- **O ponto médio do segmento de reta entre P_1 e P_3**

  $P_2 = ((X_1+X_3)/2 , (Y_1+Y_3)/2)$

- Duas retas são **perpendiculares** se e somente se
  - $M_1 = -1/M_2$
  - O cosseno do ângulo entre elas é 0.

Forma paramétrica da Reta

- O parâmetro t representa o “momento” em que a curva atinge o ponto (x,y).

Dados os pontos $P_1 = (X_1, Y_1)$ e $P_2 = (X_2, Y_2)$

$$\mathbf{X = X_1 + t(X_2 - X_1)}$$
$$\mathbf{Y = Y_1 + t(Y_2 - Y_1)}$$

t é o parâmetro.

t = 0 temos (X_1, Y_1)

t = 1 temos (X_2, Y_2)

Se $0 < t < 1$ Tem-se todos os pontos sobre o segmento de reta entre (X_1, Y_1) e (X_2, Y_2).

Exercício

- Determine a equação da reta cujo declive é 3 e passa pelo ponto $P_0(1,5)$.
- Determine a distância entre os pontos cuja as coordenadas são
  - (5,2) e (7,3); (0,1) e (2,0)
- Determine a equação da reta que passa pelos pontos $P_1(2,3)$ e $P_2(4,-2)$.
- Sejam as equações de uma reta

$$x = \frac{2+t}{1+t} \text{ e } y = \frac{4+t}{1+t}$$

  - Trace uma reta para todos os valores de t
  - Trace o segmento de reta no intervalo [0 , 8]
  - Determine o declive da reta

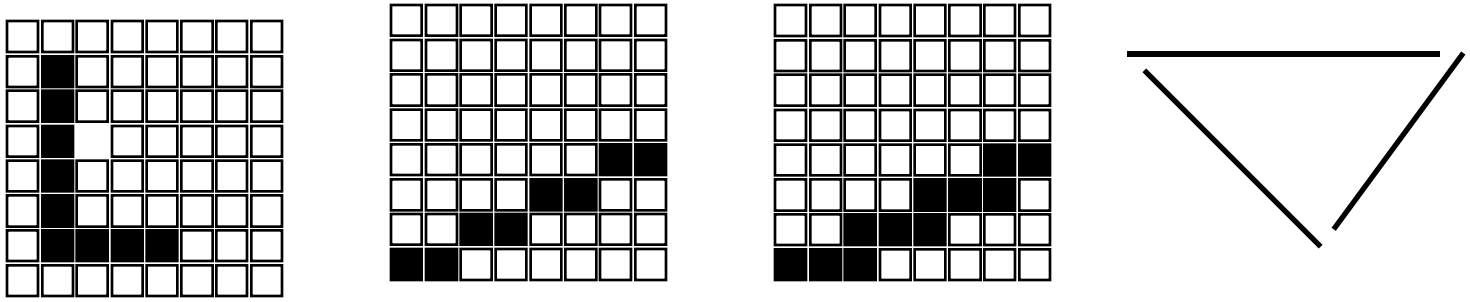
Primitivas

- Retas:
  - A mais comum primitiva 2D
  - Todo *wireframes*(aramados) 3D são eventualmente retas 2D.

  - Os algoritmos aperfeiçoados contêm numerosas técnicas e truques que ajudam a projetar algoritmos mais avançados

Características Desejáveis para Algoritmos de Rasterização da Reta

- **linearidade -** Os *pixels* traçados devem dar a aparência de que estão sobre uma reta.
- **Espessura (densidade) uniforme -** A densidade da reta é dada pelo número de *pixels* traçados dividido pelo comprimento da reta. Para manter a densidade constante, os pixels devem ser igualmente espaçados. A imagem do segmento de reta não deve variar de intensidade ou espessura ao longo de sua extensão.



Propriedades exigidas da Reta

- **precisão** - os segmentos devem iniciar e terminar nos pontos especificados. Caso isso não ocorra, pequenos *gaps* podem surgir entre o final de um segmento e o início de outro.

- **Intensidade independente da inclinação** - para segmentos de retas de diferentes inclinações.

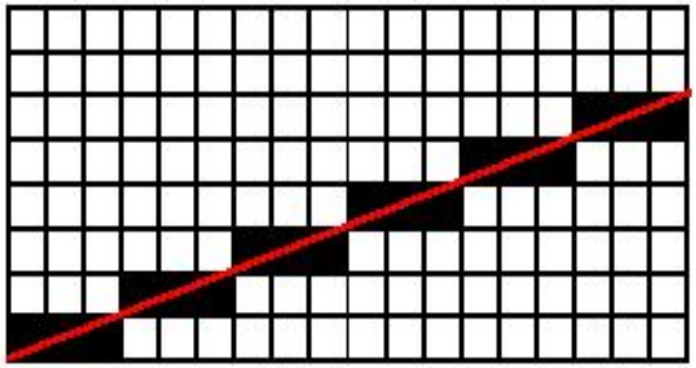
- **continuidade** - a imagem não apresenta interrupções indesejáveis.

- **rapidez no traçado dos segmentos**

Rasterização da Reta

- Tendo-se uma rotina apropriada, traçar uma reta no vídeo é simples.
- Mas a rotina não é tão simples como parece.
- Ao desenhar-se uma reta no vídeo, não se obtém um desenho perfeito e sim uma aproximação.
- A aproximação é realizada através de um plano quadriculado.

Rasterização da Reta



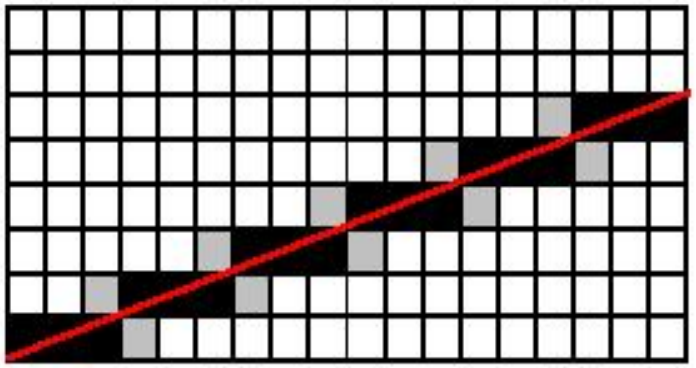
Reta Original

Aproximação feita no dispositivo raster

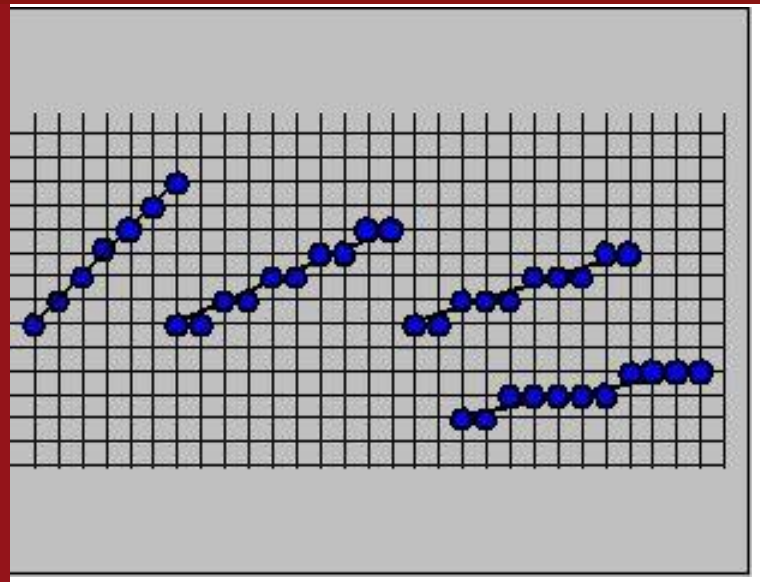
"Rasterização"

Suavização dos contornos da reta

"Anti-Aliasing"



Rasterização da Reta



- Algoritmo do declive
  - DDA : *digital diferential analyzer*
- Usa o método incremental
  - Equações para $0 \leq m \leq 1$

    $y_{i+1} = mx_{i+1} + b = m(x_i + dx) + b$ **=>**

    $y_{i+1} = y_i + mdx$, *faça* $dx = 1$ =>

    $y_{i+1} = y_i + m$
  - Equações para $m > 1$

    $x_{i+1} = x_i + \frac{1}{m}$, *faça* $dy = 1$

DDA* Simples, Algoritmo de Reta

```
void DDA(int X1,Y1,X2,Y2,valor)
{
   int    Length, I;
   float X,Y,Xinc,Yinc;

   Length = ABS(X2 - X1);
   if (ABS(Y2 - Y1)  > Length)
        Length = ABS(Y2-Y1);
   Xinc = (X2 - X1)/Length;
   Yinc = (Y2 - Y1)/Length;
```

```
   X = X1;
   Y = Y1;
setpixel(Round(X),Round(Y),valor);

   while(X<X2){
     X = X + Xinc;
     Y = Y + Yinc;

   setpixel(Round(X),Round(Y),v
   alor);
                    }
}
```

DDA Cria boas linhas, mas consome muito tempo devido as funções de arredondamento.

*DDA: *Digital Differential Analyzer* (Método do Declive)

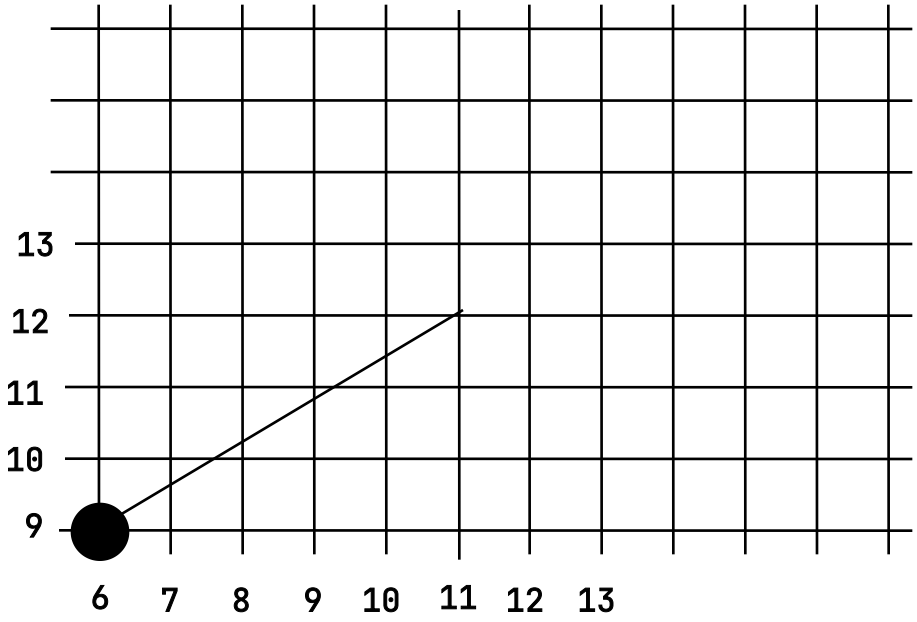
DDA COM LAÇO FOR

```
#include <stdlib.h>
#include <math.h>
 inline int round (const float a)  { return int (a + 0.5); }
 void lineDDA (int x0, int y0, int xEnd, int yEnd)
  {
     int dx = xEnd - x0,  dy = yEnd - y0,  steps,  k;
     float xIncrement, yIncrement, x = x0, y = y0;

     if (fabs (dx) > fabs (dy))
        steps = fabs (dx);
     else
        steps = fabs (dy);
     xIncrement = float (dx) / float (steps);
     yIncrement = float (dy) / float (steps);

     setPixel (round (x), round (y));
     for (k = 0; k < steps; k++) {
        x += xIncrement;
        y += yIncrement;
        setPixel (round (x), round (y));
     }
  }
```

Exemplo do DDA

Compute quais pixeis devem ser escolhidos para representar a reta de (6,9) to (11,12).

Length = ?
Xinc = ?
Yinc = ?

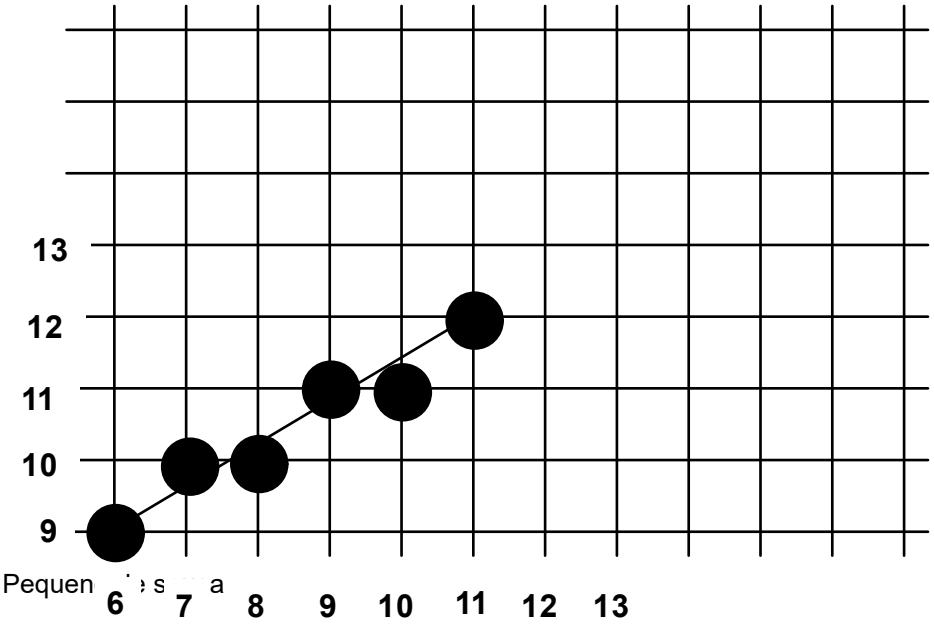


DDA Exemplo

Reta de (6,9) to (11,12).

Length := Max de (ABS(11-6), ABS(12-9)) = 5
Xinc := 1
Yinc := 0.6

Os Valores computados são:
(6,9), (7,9.6),
(8,10.2), (9,10.8),
(10,11.4), (11,12)



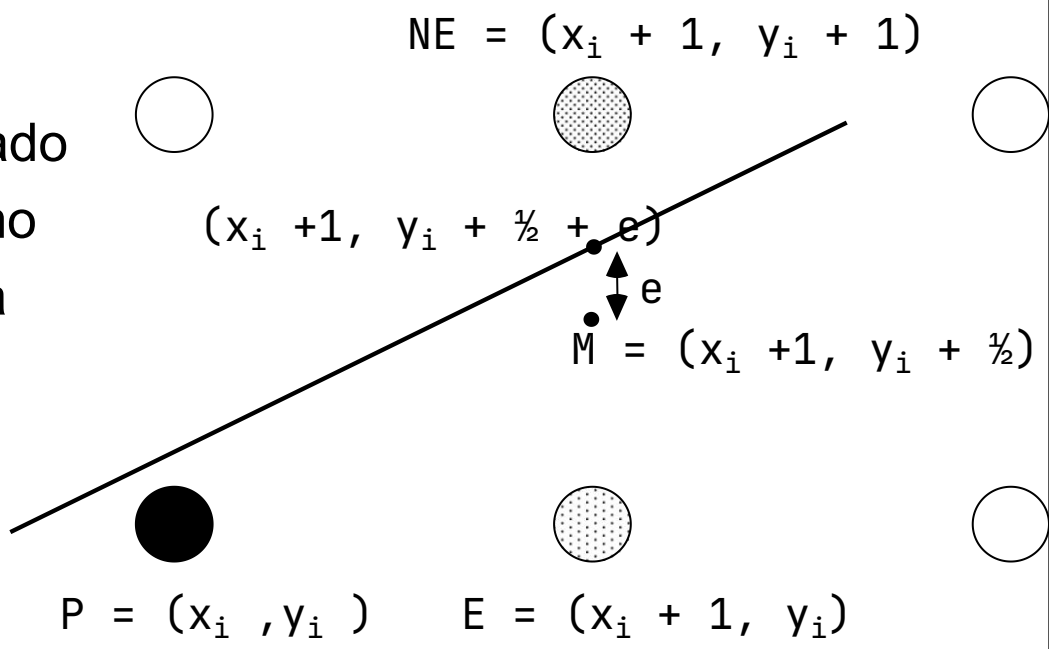
Retas Rápidas – Método do Ponto Médio

- Vamos estudar uma versão do Algoritmo de *Bresenham* (1965) denominada *Mid Point Line Algorithm* ("Algoritmo do Ponto-Médio ou Meio-Ponto para Retas") por *Foley* (2014) .

- *Bresenham* desenvolveu um algoritmo clássico que usa apenas variáveis inteiras, e permite que o cálculo de (x_i+1, y_i+1) seja feito incrementalmente, usando os cálculos já feitos para (x_i, y_i).

Retas Rápidas (cont.)

Para as retas do 1º octante, dado um pixel sobre a reta, o próximo pixel é para direita (E) ou para Direita acima (NE).



Forma Implícita a da Reta

- Vamos determinar um método para calcular de que lado da reta está do ponto M. Para isso, considere sua função implícita, $F(x,y) = ax + by + c = 0$.
  - Se $dy = y2 - y1$, e $dx = x2 - x1$,
- A equação da reta em termos de sua inclinação pode ser escrita como:

$$y = \frac{dy}{dx}x + B$$

Estudo do valor da Função

- Forma Implícita

$$F(x,y) = dy.x - dx.y + dx.B = 0$$

- Em que
  - a = dy, b = - dx e c = dx.B

- É fácil verificar que:
  - F(x,y) = 0, ponto sobre a linha
  - F(x,y) > 0, para pontos abaixo da linha
  - F(x,y) < 0, para pontos acima da linha

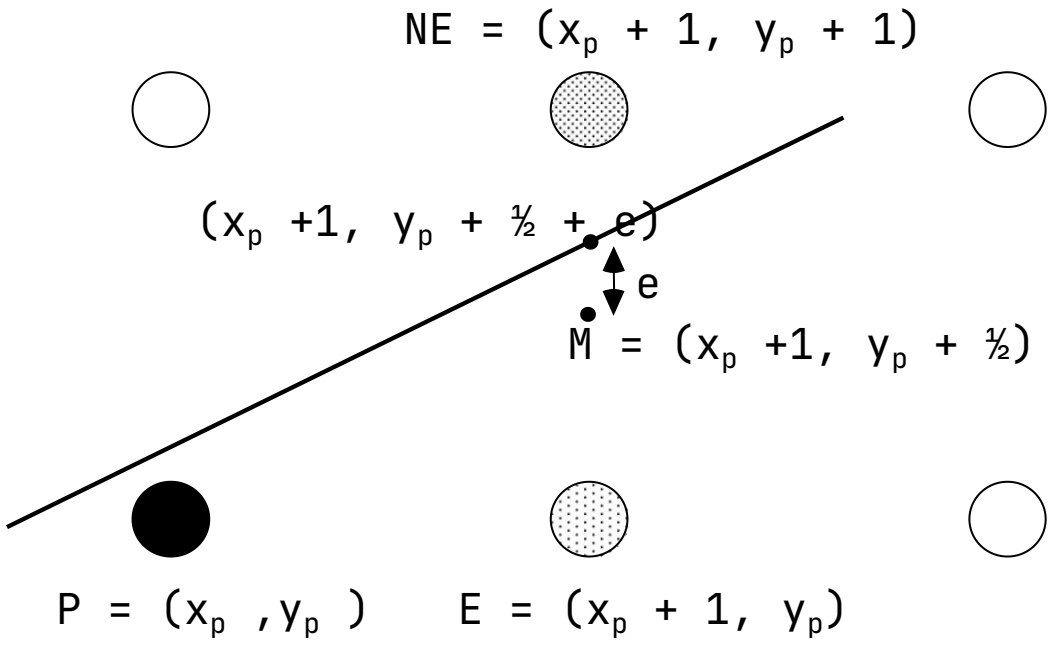
Determinando a variável de decisão

- Para o teste do ponto-médio, basta calcular $F(M) = F(x_p + 1, y_p + 1/2)$ e verificar o seu sinal.
- A decisão será tomada com base no valor da função no ponto $(x_p + 1, y_p + 1/2)$.
- define-se uma "variável de decisão"

- $d = a(x_p + 1) + b(y_p + 1/2) + c$:

  - Se $d > 0$, escolhemos o pixel NE
  - Se $d < 0$, escolhemos o pixel E
  - Se $d = 0$ pode-se escolher qualquer um deles

Escolha de E ou NE



Determinando a Forma Incremental

- Se E for escolhido, M é incrementado de 1 na direção x.

$$d_{new} = F(x_p + 2, y_p + \frac{1}{2}) = a(x_p + 2) + b(y_p + \frac{1}{2}) + c$$

$$d_{old} = a(x_p + 1) + b(y_p + \frac{1}{2}) + c$$

- Subtraindo $\boldsymbol{d_{old}}$ de $\boldsymbol{d_{new}}$ para obter a diferença incremental, tem-se
  - $\boldsymbol{d_{new} = d_{old} + a}$.

Determinando a Forma Incremental

- Se NE é escolhido, M é incrementado de 1 em ambas as direções, x e y.

$$d_{new} = F(x_p + 2, y_p + 1 + \frac{1}{2}) = a(x_p + 2) + b(y_p + \frac{3}{2}) + c$$

$$d_{old} = a(x_p + 1) + b(y_p + \frac{1}{2}) + c$$

- Subtraindo *dold* de *dnew*, tem-se

$$\boldsymbol{d_{new} = d_{old} + a + b}$$

Determinando o Ponto Inicial

- Como o primeiro pixel corresponde ao ponto *(x0,y0)*, pode-se calcular diretamente o valor inicial de d para escolher entre E e NE. O primeiro ponto-médio está em $(x_0 + 1, y_0 + 1/2)$ temos:

$$d_{start} = F(x_0 + 1, y_0 + \frac{1}{2}) = a(x_0 + 1) + b(y_0 + \frac{1}{2}) + c$$

$$d_{start} = ax_0 + a + by_0 + b.\frac{1}{2} + c = ax_0 + by_0 + c + a + \frac{1}{2}b$$

$$d_{start} = a + \frac{1}{2}b$$

- Como $F(x_0 , y_0)$ está sobre a reta, temos que

F(x0 , y0) = 0, daí o resultado acima.

Resumo das principais equações do algoritmo

- Multiplique todas as equaçoes obtidas por 2, obtendo-se assim todos os valores inteiros:

$$INC_E = 2a$$

$$INC_{NE} = 2(a+b)$$

$$d_{stard} = 2a + b$$

Algoritmo

```
void inc_line(int x1,
int y1, int x2, int y2,
int color){int dx, dy,
incE, incNE, d, x, y;

dx = x2 - x1;

dy = y2 - y1;

d = 2 * dy - dx; /*
Valor inicial de d */

incE = 2 * dy; /*
Incremento de E */

incNE = 2 * (dy - dx);
/* Incremento de NE
*/

x = x1;

y = y1;

write_pixel(x, y,
color);

while (x < x2){

    if (d <= 0){

        /* Escolhe E */

        d = d + incE;

        x = x + 1;

    }else{

        /* Escolhe NE */

        d = d + incNE;

        x = x + 1;

        y = y + 1;

        }/*, y, color); }/* end while */

        end if */

}/* end inc_line */

write_pixel(x
```

Algoritmo do ponto médio - Bresenham

```
#include <stdlib.h>
#include <math.h>

  /*  Bresenham line-drawing procedure for |m| < 1.0.  */
  void lineBres (int x0, int y0, int xEnd, int yEnd)
  {
    int dx = fabs (xEnd - x0),  dy = fabs(yEnd - y0);
    int p = 2 * dy - dx;
    int twoDy = 2 * dy,  twoDyMinusDx = 2 * (dy - dx);
    int x, y;

    /* Determine which endpoint to use as start position.  */
    if (x0 > xEnd) {
      x = xEnd;
      y = yEnd;
      xEnd = x0;
    }
    else {
      x = x0;
      y = y0;
    }
    setPixel (x, y);
```

```
while (x < xEnd) {
      x++;
      if (p < 0)
        p += twoDy;
      else {
        y++;
        p += twoDyMinusDx;
      }
      setPixel (x, y);
    }
  }
```

Exemplo

Indicar que localizações serão calculadas pelo algoritmo de Bresenham quando se gera por varredura um segmento de reta entre (20,10) e (30,18) em coordenadas de tela.

d	x	y
6	21	11
2	22	12
-2	23	12
14	24	13
10	25	14
6	26	15
2	27	16
-2	28	16
14	29	17
10	30	18

$dx = x_2 - x_1 = 10$, $dy = y_2 - y_1 = 8$

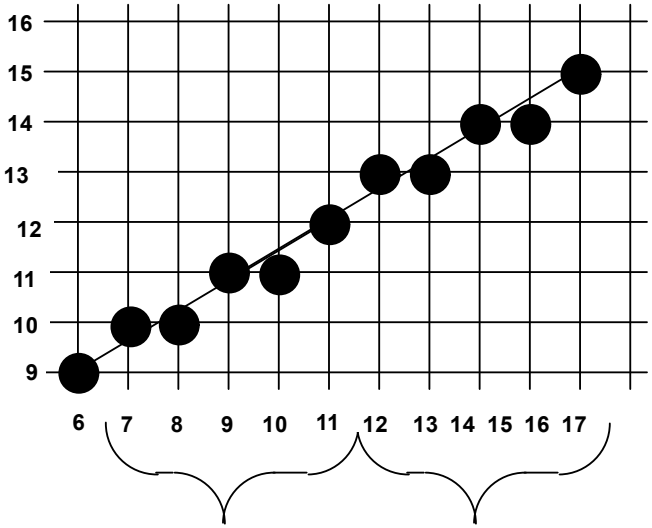
$Inc_E = 2dy = 2(y_2 - y_1) = 16$

$Inc_{NE} = 2(dy - dx) = 2[(y_2 - y_1) - (x_2 - x_1)] = -4$

$d_{start} = inc_E - dx = 16 - 10 = 6$

Otmização

- A velocidade pode ser aumentada detectando ciclos na variável de decisão **d**. Estes ciclos correspondem a padrões de pixels repetido na escolha.
- O padrão é salvo e o ciclo detectado é repetido sem recalcular.



Referências Bibliográficas

- HEARN, D., BAKER, M. P. **Computer Graphics With OpenGl**, 4. ed.: Prentice-Hall, 2011.
- FOLEY, J. et al. **Computer graphics : principles and practice**. 3. ed. Reading, MA: Addison-Wesley, 2014.
- Traima,A. J. M.; OLIVEIRA, M. C. F. **Apostila de Computação Gráfica,** Universidade de São Paulo – USP – Março de 2004.